

Encapsulation

Definition

Encapsulation is a concept in Object Oriented Programming (OOP) where we bundle data (variables) and methods (functions) into a single unit (class) and restrict direct access to the data.

NOTE: Unlike languages like Java or C++, Python does not have strict "private" or "protected" keywords. Instead, it relies on naming conventions to signal how data should be accessed.

 **Tidbit:** Hide the data and control access using methods.

Why Encapsulation is Important

- Protects data from unwanted access
- Improves security
- Makes code easier to manage
- Allows controlled modification of data

Example:

Think of a bank account:

Your balance is hidden

You can only access it using:

- deposit
- withdraw()

You cannot directly change the balance; this is encapsulation.

Levels of Access Control

In Python, you can control the visibility of class members using specific naming prefixes:

Public Members: These are accessible from anywhere—inside or outside the class.

By default, all members in Python are public.

Syntax: `self.attribute_name`

Protected Members: These are intended for internal use within the class and its subclasses. It is a convention (not a restriction) to signal to other developers that the member should not be accessed from outside.

Syntax: `self._attribute_name` (single underscore)

Private Members: These are highly restricted and cannot be easily accessed from outside the class. Python uses a mechanism called Name Mangling to change the

internal name of the variable to prevent accidental access.

Syntax: self.__attribute_name (double underscore)

Type	Syntax	Access Level
Public	self.attribute	Accessible from anywhere
Protected	self._attribute	Intended for class & subclasses (convention)
Private	self.__attribute	Restricted via Name Mangling

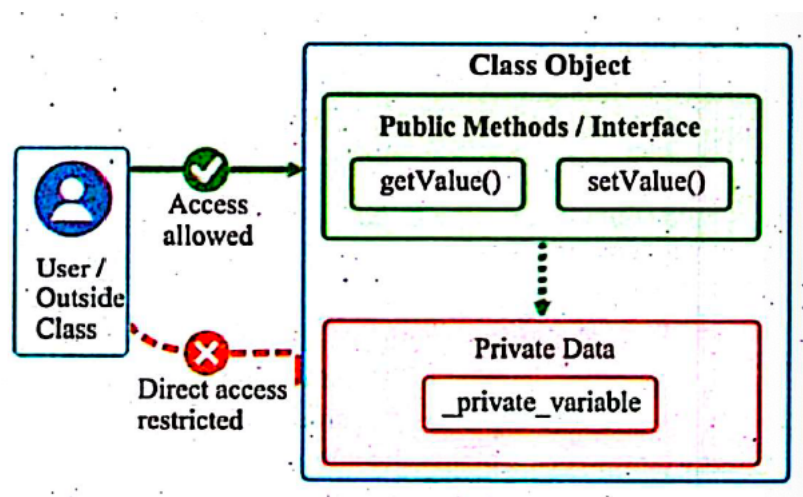


Figure 3.2: Encapsulation diagram showing private and public variables

Example – Bank Account (Private Balance)

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    #Private
variable

    def deposit(self, amount):
        self.__balance += amount

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Insufficient balance")
```

```
def get_balance(self):  
    return self.__balance  
  
account = BankAccount(1000)  
account.deposit(500)  
account.withdraw(300)  
print(account.get_balance())    # Output: 1200  
# print(account.__balance)      # Error - direct  
# access not allowed
```

Short Questions

1. What is encapsulation?
2. Why is encapsulation important?
3. What are the levels of access control in Python?
4. How do we make a variable private in Python?
5. What is name mangling?

Long Questions

1. What is encapsulation in OOP? Explain its importance and levels of access control in Python with example.
2. Write a Python program for a BankAccount class that uses encapsulation (private balance) with deposit, withdraw and get_balance methods.